

Friemi v2.7 交互延迟与并发容量分析

日期：2026-08-12

分支：perf/v2-7-concurrency-scalability-analysis

状态：静态分析、平台容量研究和两轮低风险请求降载完成，预览环境压测与容量验收待执行

1. 问题定义

当前问题不是单一页面动画不流畅，而是用户数量增加、多人同时操作时，页面请求、认证、数据库查询和前端刷新互相放大，最终表现为：

- 点击后等待时间明显增长。
- 聊天消息、未读数、在线状态和游戏状态更新变慢。
- 页面切换或 `router.refresh()` 长时间处于等待状态。
- 高峰期可能出现 Prisma P2024、请求超时、空白页或局部数据加载失败。
- 单人测试正常，但多人同时在线时延迟非线性上升。

这类问题不能只靠调整 loading 动画或提高服务器规格解决。必须先控制单个用户产生的后台请求数量，再降低每个请求的数据库成本，最后验证连接池和部署容量。

2. 本次分析范围

已检查

- Next.js 页面与 API 的动态渲染范围。
- `router.refresh()`、`revalidatePath()` 和定时轮询。
- 私聊、聚吧群聊、通知角标、在线状态和狼人杀同步链路。
- Prisma 连接配置、查询聚合方式和主要索引。
- 当前性能埋点、部署区域和自动化测试能力。
- 首页、活动、聚吧、搜索、Profile 和消息查询模块的静态复杂度。

尚未完成

- 预览环境 20 / 50 / 100 并发用户压测。
- Supabase 高峰期连接数、CPU、I/O、锁等待和慢查询采样。
- Vercel Function P50 / P95 / P99、冷启动率和并发实例数据。
- 真机 Android WebView 的 INP、长任务、内存和图片解码记录。

因此，本文可以确认代码层存在明显的并发放大机制，但在压测完成前，不给出“系统最多支持多少用户”的虚假结论。

3. 结论摘要

核心判断

当前最可能的主瓶颈顺序如下：

1. **后台轮询形成固定请求税**：用户即使没有主动操作，也会持续请求未读数和在线状态。
2. **聊天使用路由级刷新**：私聊每 6 秒、聚吧群聊每 8 秒重新执行当前路由的 Server Component 数据树，而不是只获取增量消息。
3. **狼人杀同步频率随人数线性放大**：大厅每位玩家约每 1.8 至 2.7 秒请求一次同步版本。
4. **单实例 Prisma 连接上限为 1**：同一个 Function 实例内的并行 Prisma 调用会竞争同一数据库连接，复杂页面容易排队。

5. **动态 SSR 比例高**：76 个页面中有 41 个显式 force-dynamic，多数访问都需要重新认证和读取数据库。
6. **热点页面查询聚合较重**：活动、聚吧和 Profile 查询文件体积大、查询分支多，用户并发会直接转化为更多数据库工作。
7. **监控本身写入主数据库**：页面和操作延迟事件仍写入同一个 PostgreSQL，数据库繁忙时会与业务请求竞争连接。

不是主要结论的内容

- findMany 数量多不等于每个查询都有问题，必须结合 take、索引和 EXPLAIN ANALYZE 判断。
- Promise.all 只表示 JavaScript 并发，不保证数据库查询真正并行；当前 connection_limit=1 下尤其不能把它当作性能优化。
- 增大连接池不一定能解决问题。若请求总量没有下降，盲目增加连接可能把排队从应用层转移到 PostgreSQL，反而加重数据库负载。
- UI 图片和 React 重渲染会影响低端设备，但无法解释服务端在多人同时操作时普遍变慢，应该放在服务端瓶颈之后处理。

4. 已确认的代码证据

项目	当前事实	影响	置信度
Prisma 连接	DATABASE_URL 使用 Supabase transaction pooler，端口 6543，pgbouncer=true，connection_limit=1	每个运行实例最多使用一个 Prisma 数据库连接，请求内查询容易排队	已确认
部署区域	Vercel 为 cdg1，数据库位于 AWS eu-west-1	每次数据库往返存在跨区域延迟，查询数量越多放大越明显	已确认
动态页面	76 个页面中 41 个显式 force-dynamic	页面访问通常不能直接命中静态页面缓存	已确认
动态 API	36 个 API route 中 28 个显式 force-dynamic	轮询接口每次都会进入 Function、认证并访问数据源	已确认
路由刷新	25 个文件共 47 处 router.refresh()	会重新请求并执行当前路由的 RSC 数据树	已确认
缓存失效	29 个文件共 168 处 revalidatePath()	写操作可能触发多个页面失效，扩大后续读取成本	已确认
定时器	10 个文件共 12 处 setInterval()	其中消息、角标、presence 和游戏定时器会产生网络请求	已确认
Prisma 查询	app 和 features 中共 161 处 findMany()	数据增长后存在分页、返回字段和索引风险，需逐项审计	已确认，不能直接判定为缺陷
性能埋点	12 个页面/组件使用 page tracker，7 个文件使用 action tracker	已有局部耗时证据，但缺少统一 API、SQL、连接池和客户端指标	已确认
压测工具	项目有 Playwright monitoring，但没有 k6、Artillery 或同类容量测试脚本	Playwright 能做体验回归，不能替代并发容量测试	已确认

关键代码位置：

- apps/web/lib/prisma.ts
- apps/web/lib/performance.ts

- apps/web/features/notifications/components/NotificationBadgeProvider.tsx
- apps/web/features/direct-messages/components/MessageThreadAutoRefresh.tsx
- apps/web/features/activity-room-chat/components/ActivityRoomChatPage.tsx
- apps/web/features/profile/components/PresenceHeartbeat.tsx
- apps/web/features/game-tools/components/WerewolfRoomOverview.tsx
- apps/web/app/api/game-tools/werewolf/rooms/[roomId]/sync/route.ts
- apps/web/features/analytics/server.ts

5. 请求放大模型

以下估算只计算当前源码中明确存在的周期请求，不包含用户点击、图片、页面跳转、Server Action 和第三方认证请求。

5.1 登录用户停留在普通页面（第一轮优化前基线）

第一轮优化前，生产环境通知 Provider 每 15 秒请求：

- /api/notifications/unread-count
- /api/direct-messages/unread-count

即每位可见页面用户每分钟产生：

- 4 次通知未读请求。
- 4 次消息未读请求。
- 合计 8 次 GET。
- Presence 当时每 60 秒至少 1 次 POST，切回前台还会额外触发。

因此，100 名登录用户只是保持页面可见、不做任何操作，每分钟也会产生约 800 次未读 GET 和 100 次以上 presence 写入。

消息未读接口不是单次轻查询。当前可见路径至少包含：

- 查询当前 UserProfile。
- 查询免打扰会话。
- 统计私聊未读。
- 查询最多 100 个可访问聚吧。
- 使用最多 100 组 OR 条件聚合群聊未读。

按源码中的独立 Prisma 调用估算，仅两类未读接口就可能达到约 24 次数据库操作/用户/分钟，即 100 名空闲用户约 2,400 次数据库操作/分钟。实际数量需要 Prisma query 日志验证。

5.2 私聊页面

MessageThreadAutoRefresh 默认每 6 秒执行一次 router.refresh()：

- 每位停留在私聊页面的用户约 10 次路由刷新/分钟。
- 20 名活跃聊天用户约 200 次路由刷新/分钟。
- 每次刷新会重新执行用户读取、会话读取、已读写入、私聊未读统计、群聊未读统计和性能记录等逻辑。

这里的 router.refresh() 不是浏览器整页刷新，但它会重新获取并执行当前路由的 Server Component 数据树，成本远高于“只请求最新消息”。

5.3 聚吧群聊页面

ActivityRoomChatAutoRefresh 默认每 8 秒执行一次 router.refresh()：

- 每位用户约 7.5 次路由刷新/分钟。

- 20 人群聊约 150 次路由刷新/分钟。
- 发送、邀请、公告和管理操作成功后还有额外刷新。

页面查询除了消息，还可能包含活动权限、成员、管理员、公告、签到和未读状态，因此用户越多时，刷新成本和更新冲突都会增加。

5.4 狼人杀房间

普通设备的同步间隔：

- 大厅：1.8 秒，加 0 至 0.9 秒随机抖动。
- 游戏中：4.2 秒，加 0 至 0.9 秒随机抖动。

一个 12 人大厅大约产生 267 至 400 次 sync probe/分钟，平均约 320 次。10 个同时活跃房间会达到约 3,200 次 probe/分钟。

每个 probe 当前查询：

- GameToolRoom 状态和更新时间。
- 最新一条 GameToolEvent。

检测到版本变化后，每个客户端还会再次请求完整房间，完整返回成员、座位和最近 30 条事件。虽然已有 in-flight 防重、页面隐藏暂停、jitter 和低端设备降频，但跨设备之间没有真正的服务端事件推送，因此人数仍会线性增加请求量。

6. 数据库与查询风险

6.1 connection_limit=1 是当前最重要的容量约束之一

该配置适合防止 serverless 实例创建过多 PostgreSQL 连接，但也意味着：

- 同一实例中的多个 Prisma 查询需要排队。
- 页面中大量 Promise.all() 不一定能缩短数据库阶段耗时。
- 单个慢查询会阻塞该实例后续查询，形成 head-of-line blocking。
- 并发升高后，更多 Function 实例会各自建立连接，最终仍可能触碰 Supabase pooler 或数据库上限。

不能直接把 connection_limit 改大。正确顺序是：

1. 降低轮询和每请求查询数。
2. 获取真实 pool 使用率与 P2024 频率。
3. 在 20 / 50 / 100 并发阶梯压测下比较连接数 1、2、3 的结果。
4. 确认数据库连接上限和 Vercel 最大并发后再调整。

6.2 未读统计会随历史数据增长

私聊未读当前先读取免打扰会话 ID，再使用 notIn 排除并统计消息。随着会话数量和消息历史增长：

- notIn 参数会增长。
- readAt IS NULL、senderId != currentUser 和 conversation 关系条件需要稳定查询计划。
- 当前索引不一定完全覆盖该全局统计路径，需要用真实参数执行 EXPLAIN (ANALYZE, BUFFERS)。

群聊未读当前最多读取 100 个房间，并把每个房间的 lastReadAt 展开成 OR 条件后 groupBy。这比单房间未读复杂得多；优化前由两个独立接口每 15 秒触发，低风险两轮完成后由聚合接口约每 45 至 50 秒触发一次。

优先方案不是继续增加索引，而是建立按用户/会话维护的未读计数或最后消息序号，使角标读取变成小范围定点查询。

6.3 热点服务文件复杂度高

静态统计如下：

文件	行数	findMany	count	Promise.all
getActivities.ts	3,146	21	6	13
getActivityLobby.ts	1,478	20	2	13
getProfileDashboard.ts	1,144	15	8	4
getGlobalSearchResults.ts	940	8	0	6
getDirectMessages.ts	942	5	1	6

这些数字不直接代表 N+1，但说明一个页面调用可能经过多条条件分支和装饰查询。需要针对实际入口记录：

- 每个请求执行了多少条 SQL。
- 最慢 SQL 及其 query plan。
- 返回行数、返回字节和序列化耗时。
- 相同请求内是否重复读取 UserProfile、关注关系、收藏或未读数。

6.4 动态 SSR 和个性化数据耦合

大量页面把公共内容和用户状态一起放在动态路由中。例如活动公共卡片数据本可短时缓存，但收藏、参与状态和未读状态使整个页面动态化。

目标应拆为：

- 公共活动、分类、Top News：带 tag 的短时服务端缓存。
- 当前用户收藏、参加状态：小型个性化 overlay。
- 聊天和游戏状态：增量实时接口。
- 管理页面：保持动态，但不参与全局预取。

6.5 分析事件与业务共用数据库

queueAnalyticsEvent 使用进程内 Promise 队列，最终逐条写入 AnalyticsEvent：

- 队列只在单个实例内有效，不是跨实例消息队列。
- 每次页面性能记录仍会消耗主库连接和一次 INSERT。
- 代码已经专门捕获 Prisma P2024 并在 pool busy 时跳过事件，说明设计者已遇到或预期连接池争用。
- 高峰期应该采样、批处理或发送到独立观测系统，不能让性能监控成为业务数据库压力来源。

7. 前端交互慢的两种来源

服务端等待

特征：

- 点击后 loading 很久，Network 中 RSC/API TTFB 高。
- 多人同时操作时所有设备一起变慢。
- Vercel Function duration、Prisma query 和连接等待同步升高。

主要对应本文前述的轮询、动态 SSR、连接池和查询问题。

客户端主线程阻塞

特征：

- 请求已经返回，但点击、滚动或输入仍卡顿。
- Chrome Performance 出现超过 50ms 的 long task。
- React Profiler 显示整个列表、桌面或聊天消息重复渲染。
- 图片解码、模糊背景或大量动画期间更明显。

客户端优化仍需要做，但必须以 INP、long task 和 commit duration 为证据。不能把服务端 TTFB 问题误判为 React 动画问题。

8. 目标架构

8.1 消息与游戏使用增量同步

短期不必立即建设完整 WebSocket 平台，可以先统一为增量拉取：

- 私聊：GET /messages?conversationId=...&after=<messageId>。
- 群聊：GET /room/messages?activityId=...&after=<messageId>。
- 狼人杀：用单调递增 revision 判断是否变化，变化后返回 delta 或一次完整快照。
- 响应支持 ETag / If-None-Match 或明确的 revision，未变化时不执行完整查询。
- 发送成功直接返回标准 ViewModel，客户端 optimistic append，不再立即 router.refresh()。
- 页面隐藏、离线和省流模式暂停；失败后指数退避并保留 jitter。

中期再根据真实同时在线人数，在 SSE、WebSocket、Supabase Realtime 或托管 pub/sub 中选择一种。选择依据是连接数、移动端后台行为、费用和故障降级，不按技术偏好决定。

8.2 未读数改为事件驱动

- 通知和消息角标合并为一个 endpoint 或一次事件 payload。
- 发送消息时更新收件人的 conversation unread state。
- 已读时按 conversation 定点清零或推进 last-read sequence。
- 免打扰会话保留红点，但不进入总数，读取时不再构建不断增长的 notIn。
- 客户端收到新事件后本地增加角标，页面聚焦时做低频校准。

8.3 Presence 降低写放大

- 优化前每 60 秒更新一次 UserProfile，会产生持续主表写入和索引更新；低风险第二轮已调整为 90 秒。
- 短期可提高心跳间隔，并只在状态转换或最后更新时间超过阈值时写入。
- 中高并发阶段使用带 TTL 的 ephemeral store 保存在线状态，PostgreSQL 仅保存最后活跃时间的低频快照。
- 离线事件继续 best effort，最终状态由 TTL 保证，而不是依赖 beforeunload 一定成功。

8.4 公共数据缓存与个性化 overlay

- 为活动、公开活动、分类和 Top News 建立 revalidateTag 缓存边界。
- 创建/编辑操作只失效对应实体和列表 tag，减少一次操作调用多个 revalidatePath()。
- 用户收藏、关注、参与和未读状态使用轻量查询叠加。
- 避免在全局 layout 中阻塞读取非首屏必需数据。

8.5 数据库连接和区域

- 保持 pooler，但用压测确认每实例合理的 connection_limit。

- 确认 Supabase 项目可用连接数、pool mode 和等待超时。
- 尽可能让计算和数据库位于同一云区域；若无法完全一致，减少每次请求的 SQL round trip 数。
- 对慢查询使用真实参数执行 EXPLAIN (ANALYZE, BUFFERS) 后再补索引。
- 为高频读取建立小型 read model，不让角标和列表页反复扫描历史消息。

9. 分阶段落实方案

P0：建立证据并停止请求风暴

目标：先让系统在现有数据库规格下不因固定轮询失控，同时获得可信基线。

- 为关键 API、Server Action 增加统一 request ID、总耗时、DB query count、DB duration 和响应大小。
- Prisma query 监控只在预览环境采样开启，避免生产全量 SQL 日志泄露参数或增加开销。
- 在 Vercel 和 Supabase 同一时间窗口记录 Function P95/P99、连接数、P2024、CPU、I/O 和锁等待。
- 合并两个全局未读轮询请求，并改为 30 至 60 秒校准、focus 触发和事件后本地更新。
- 私聊和群聊停止周期性 router.refresh()，先改为只拉取增量消息。
- 狼人杀 probe 改为单字段 revision 查询，并根据活跃度自适应降频。
- 将性能 AnalyticsEvent 改为采样，连接池繁忙时必须无等待丢弃。
- 建立可重复的 k6 或 Artillery 场景，Playwright 继续负责页面正确性而不是容量。

预期：100 名登录用户保持页面可见时，后台固定请求量比当前下降至少 70%，且不出现 P2024。

P1：降低单请求数据库成本

目标：减少复杂 SSR 和角标查询对数据库的重复读取。

- 将通知、私聊、群聊未读聚合为用户级 read model。
- 审计 getActivities.ts、getActivityLobby.ts、getProfileDashboard.ts 和搜索查询。
- 所有列表确认 cursor/limit、稳定排序和最大返回字节。
- 对实际慢 SQL 执行 EXPLAIN ANALYZE，补充必要的复合或 partial index。
- 将公共活动和首页数据缓存，用户态信息单独加载。
- 缩小 revalidatePath() 范围，改为实体和列表 tag。
- 消除同一请求内重复 UserProfile、关注关系、收藏和未读读取。

预期：关键读接口 DB query count 和 DB duration P95 至少下降 40%。

P2：实时通道与写路径治理

目标：多人消息和游戏从轮询模型转为事件模型。

- 选择并落地统一实时通道，覆盖私聊、群聊、通知和狼人杀 room revision。
- 服务端写操作发布小型事件，客户端只更新受影响状态。
- 建立断线重连、last event ID、幂等、乱序和漏事件校准机制。
- Presence 迁移到 TTL 模型，减少 UserProfile 高频写。
- Analytics 使用独立队列/服务或批量落库，不与业务请求竞争连接。

预期：实时消息延迟不依赖固定轮询间隔，活跃用户增加时请求数不再按“人数 × 轮询频率”线性失控。

P3：客户端渲染和资源优化

目标：服务端稳定后，解决低端手机上的主线程卡顿。

- 用 React Profiler 检查聊天列表、狼人杀座位、活动卡片和 Profile 面板。
- 消息和长列表使用 cursor 分页；达到实际长度阈值后再引入虚拟列表。
- 稳定 key、memo 边界和 selector，避免单条消息更新重绘整页。
- 图片使用明确尺寸、缩略图、懒加载和解码失败 fallback。
- 动效仅使用 transform/opacity，并尊重 prefers-reduced-motion。
- 在 Android WebView 真机记录 INP、long task、内存峰值和滚动帧率。

预期：请求已完成后，页面输入、点击和滚动不再因大列表或整桌重渲染卡顿。

10. 压测设计

压测只能在专用预览环境和测试账号执行。禁止直接对生产环境进行写压测，禁止复用真实用户 token。

场景	并发	时长	行为	主要观察
登录空	20 / 50 / 100	每档 10 分钟	保持普通页面可见	固定请求率、连接数、presence 写入、P2024
闲基线	20 / 50 / 100	每档 10 分钟	首页、活动、聚吧、搜索、Profile 按权重切换	TTFB、RSC 大小、查询数、缓存命中
混合页面浏览	10 / 20 / 50 对	10 分钟	每人每 5 至 15 秒发送，持续接收和已读	发送确认、接收延迟、重复/漏消息
私聊	20 / 50 人单群	10 分钟	并发发消息、公告、已读	热点行、锁等待、群聊增量同步
聚吧群聊	1 / 5 / 10 个	10 分钟	入座、准备、退出、重新加入	sync RPS、room revision、完整快照次数
狼人杀大厅	12 人房	15 分钟	阶段切换、投票、死亡、重连	写入冲突、事件顺序、端到端延迟
狼人杀游戏中	12 人房	5 分钟	多图上传，不包含超大恶意文件	Function 内存、上传耗时、外部存储吞吐
照片上传	5 / 10 / 20			

每一档必须遵循：

1. 先空载 5 分钟。
2. 逐级升压，不从 0 直接跳到 100。
3. 每档结束保留 5 分钟恢复窗口。
4. 保存测试版本、环境变量指纹、数据规模和时间窗口。
5. 同时导出客户端、Vercel、Supabase 和应用性能日志。
6. 测试后清理测试消息、房间、通知和上传文件。

11. 暂定 SLO 与验收标准

这些阈值是第一版工程目标，完成首轮基线后可以根据业务和套餐能力调整。

指标	目标
公共/登录页面 TTFB	P95 < 1.5s, P99 < 3s
普通读取 API	P95 < 400ms, P99 < 1s
业务写操作	P95 < 800ms, P99 < 1.5s
聊天发送到服务端确认	P95 < 800ms
消息/游戏事件到其他在线客户端	P95 < 1.5s
API 5xx	< 0.5%
Prisma P2024	0
DB pool 使用率	稳态 < 70%，短峰值 < 85%

指标	目标
客户端 INP	P75 < 200ms
单次 React long task	高频操作期间不持续出现 > 50ms 长任务
数据正确性	不丢消息、不重复扣款、不重复入座、不越权展示角色

容量验收必须同时满足延迟、错误率和数据正确性。仅平均响应时间好看，或仅页面最终能打开，都不能算通过。

12. Checklist

分支与分析

- ☒ 创建独立性能分析分支。
- ☒ 检查 Prisma 连接和部署区域配置。
- ☒ 统计动态页面、动态 API、轮询、路由刷新和缓存失效调用。
- ☒ 检查通知、私聊、群聊、presence 和狼人杀请求链路。
- ☒ 建立当前请求放大模型。
- ☒ 明确 P0 至 P3 优化边界。

P0

- ☐ 建立统一 request ID 和 API/Action 耗时日志。
- ☐ 增加采样式 Prisma query count/duration 记录。
- ☐ 建立 Vercel 与 Supabase 同时间窗口监控表。
- ☒ 合并并降低全局未读轮询。
- ☐ 私聊替换周期性 router.refresh()。
- ☐ 聚吧群聊替换周期性 router.refresh()。
- ☐ 狼人杀改用单调 revision 和自适应同步。
- ☐ AnalyticsEvent 采样或迁出关键链路。
- ☐ 新增可重复的并发压测脚本。
- ☐ 完成 20 / 50 / 100 用户基线并记录结果。

第一层级实施记录：全局未读请求收敛

本轮只调整请求层，不修改未读定义、聊天刷新周期、狼人杀同步、数据库结构或页面布局。

- ☒ 新增 /api/navigation/unread-counts，一次认证和一次 Profile 查询后返回通知、私聊、聚吧群聊及聊聊总未读数。
- ☒ NotificationBadgeProvider 从每轮两个 GET 收敛为一个 GET。
- ☒ Provider 对同一时刻由路由、focus、visibility 和定时器触发的刷新做 in-flight 去重。
- ☒ 保留原有两个未读接口；新接口在 404/405 时回退旧接口，便于滚动发布和快速回退。
- ☒ 聚合接口 401 时清零，503 或网络失败时保留最后一次有效角标，不把服务故障伪装成零未读。
- ☒ 聚合接口响应增加 x-request-id 和 Server-Timing，失败、慢请求和预览环境采样请求输出结构化日志。
- ☒ 不信任响应中的聊聊总数，客户端用私聊数与聚吧群聊数重新计算，保持现有角标语义。
- ☒ 新增 payload 校验、角标合并和请求指标单元测试。
- ☒ 将 request ID 和耗时指标扩展到 Presence 与狼人杀同步 API。
- ☐ 将 request ID 和耗时指标继续扩展到后续聊天增量 API/Action。
- ☒ 将生产角标校准间隔从 15 秒改为 45 秒，并以 focus/业务事件保证及时性。
- ☐ 建立用户级未读 read model，降低当前聚吧房间扫描和 OR 聚合成本。

本轮静态预期：生产环境每位持续可见用户的全局角标请求由每分钟 8 个 GET 降为 4 个 GET，HTTP 请求和 Function 调用约减少 50%。认证链路由每轮两次减少为一次；底层未读统计 SQL 仍基本不

变，因此本轮不能宣称数据库负载下降 50%，必须在预览环境用相同流量窗口验证 Function 调用数、P95/P99 和数据库耗时。

回退条件：若聚合接口的错误率、P95 或角标准确性劣于旧接口，客户端可暂时恢复两个旧路径；本轮没有 migration 和数据回填，回退不涉及数据库操作。

第二层级实施记录：低风险降频与暂停

本轮只落实风险评级为低或极低的优化，不修改私聊/群聊消息协议、狼人杀同步逻辑、缓存、索引、Prisma schema 或数据库连接配置。

- ☒ 生产环境全局角标校准间隔从 15 秒调整为 45 秒，开发环境保持 60 秒。
- ☒ 使用递归 setTimeout 替换固定 setInterval，下一次刷新只在上一次请求结束后安排。
- ☒ 页面隐藏或网络离线时清除角标定时器，不产生后台请求。
- ☒ 页面重新可见、网络恢复、窗口 focus 和现有业务事件继续立即校准。
- ☒ 失败后按约 45 / 90 / 180 秒退避，最大 5 分钟；每次正常轮询增加最多 5 秒随机抖动。
- ☒ 401 继续作为已处理状态清零；503、无效 payload 和网络错误保留最后有效值并进入退避。
- ☒ Presence 心跳从 60 秒调整为 90 秒，原有后台暂停、离线上报和前台立即恢复逻辑保持不变。
- ☒ Presence 与狼人杀同步 API 增加 request ID、Server-Timing、慢请求和预览环境低比例采样日志。
- ☒ 新增轮询间隔、失败退避、上限和抖动单元测试。
- ☐ 在预览环境记录修改前后同一时间窗口的请求数、Function P95/P99、Presence 写入数和数据库耗时。

本轮静态预期：持续可见且没有额外业务事件时，每位用户的全局角标 GET 从最初每分钟 8 次降到约 1.33 次，相比原始实现减少约 83%，相比第一层实现减少约 67%。Presence 从每分钟约 1 次写入降到约 0.67 次，减少约 33%。100 名持续在线用户对应的固定角标 GET 由约 800 次/分钟降到约 133 次/分钟，Presence 写入由约 100 次/分钟降到约 67 次/分钟。

保留风险：被动角标在没有业务事件时最多约 45 至 50 秒后校准；浏览器恢复事件若被系统丢弃，会等待下一轮校准。消息本身、未读记录和游戏状态不受影响。所有间隔均为代码常量，本轮没有 migration，回退只需恢复间隔和调度器。

P1

- ☐ 建立用户级未读 read model。
- ☐ 审计四个热点聚合查询模块。
- ☐ 对真实慢 SQL 执行 EXPLAIN (ANALYZE, BUFFERS)。
- ☐ 根据 query plan 增加或调整索引。
- ☐ 公共数据缓存与用户 overlay 拆分。
- ☐ 将广泛 path revalidation 收敛为 tag invalidation。
- ☐ 所有长列表落实 cursor/limit 和 payload 上限。

P2

- ☐ 选定实时传输方案并记录容量/费用/降级决策。
- ☐ 私聊、群聊、通知、狼人杀统一事件协议。
- ☐ 完成断线恢复、幂等、乱序和漏事件校准。
- ☐ Presence 使用 TTL 或低频持久化。
- ☐ Analytics 与业务主库连接解耦。

P3

- ☐ 完成高频页面 React Profiler 记录。
- ☐ 完成长列表分页和必要的虚拟化。

- 完成图片解码、缩略图和失败降级检查。
- 完成 Android WebView 真机 INP、long task 和内存检查。

最终验收

- 所有暂定 SLO 在 100 并发混合场景下通过。
- 10 个 12 人狼人杀房间同时运行无持续卡顿。
- 50 人聚吧群聊无丢消息、重复消息或明显刷新跳动。
- 数据库无 P2024，连接数和 CPU 在安全水位内。
- 负载停止后系统能在 5 分钟内恢复基线。
- 压测报告包含失败点、修复前后对比和原始指标来源。

13. 实施约束

- 不在没有基线的情况下大规模重写聊天或游戏。
- 不把“增加 loading”当作性能修复。
- 不通过隐藏错误、吞掉业务失败或降低数据一致性换取延迟数字。
- 不直接在生产环境进行写压测。
- 不在同一个 PR 同时修改连接池、查询、实时协议和 UI，避免无法判断收益来源。
- 每一阶段都保留 feature flag 或快速回退路径。
- 数据库 migration 必须先独立环境验证，并记录回填和回滚成本。

14. 推荐执行顺序

1. 先完成 P0 观测和压测脚本，保存未优化基线。
2. 优先处理全局未读轮询、私聊/群聊整路由刷新和狼人杀 probe。
3. 用相同脚本复测，确认请求率、P95/P99 和 P2024 的变化。
4. 再根据真实慢 SQL 做 P1，不按静态命中数量盲目加索引。
5. 当增量轮询仍不能满足目标容量时，再进入 P2 实时通道。
6. 服务端 SLO 稳定后执行 P3 真机渲染优化。

第一轮实施不建议先改字体、布局或页面视觉。当前卡顿的主要风险位于请求模型和数据库排队，视觉改动无法降低并发压力。

15. 免费套餐是不是当前卡顿的主要原因

结论

免费套餐可能放大冷启动、CPU、内存、连接和配额问题，但目前没有证据证明 Vercel Hobby 本身正在排队。当前源码已经存在无需等待平台指标也能确认的请求放大，因此不能把问题全部归因于“免费平台拥挤”。

更准确的判断是：

- **Supabase Nano 的资源较小**，复杂查询和并发数据库操作更容易达到 CPU、内存或 I/O 上限。
- **Vercel Function 会自动扩展**，Hobby 和 Pro 都具备自动并发扩展；Pro 提高的是可用资源、付费扩展、观测和上限，不会自动减少应用发出的请求。
- **当前 Prisma 主动限制为每实例一个连接**。即使数据库升级，`connection_limit=1` 仍会让同一实例中的数据库工作排队，除非经过压测后调整。
- **免费升级到 Pro Micro 不代表连接上限大幅提升**。Supabase 官方表中 Nano 和 Micro 都是 60 个 direct connections、200 个 pooler clients；Micro 的主要提升是付费生产能力、CPU/内存和平台功能。Small 才提高到 90/400。
- **当前聊天和游戏轮询会消耗任何套餐**。升级后可能暂时更快，但请求数量、费用和数据库工作仍随在线人数线性增长。

因此，“先充值再观察”不能替代架构治理；“为了避免以后迁移，现在立刻自建全部基础设施”同样不合理。

另外，Vercel 当前将 Hobby 定位为个人、非商业用途。如果 Friemi 已进入商业运营，升级 Pro 首先是计划合规和生产支持决策，即使性能测试暂时没有触顶，也不应长期依赖 Hobby。

官方容量资料：

- [Vercel Functions concurrency scaling](#)
- [Vercel Fluid Compute](#)
- [Supabase Compute and Disk](#)
- [Supabase Compute usage and pricing](#)
- [Supabase Production Checklist](#)

16. Pro 能不能让大量用户都用得舒服

不能只用“Pro”这个名称回答。需要同时明确：

- 注册用户数（Registered Users）。
- 月活（MAU）和日活（DAU）。
- 峰值同时在线（CCU）。
- 峰值 HTTP RPS。
- 峰值消息发送数和分发数/秒。
- 单条事件平均 fan-out 人数。
- 数据库读写比例和最慢 SQL。
- 图片上传/下载带宽和 CDN 命中率。

10 万注册用户可能只有 200 人同时在线，也可能在一次活动中有 2 万人同时在线。这两者虽然注册数相同，架构需求完全不同。

Vercel 的实际边界

Vercel 官方当前说明：

- Function 在所有计划上自动扩展。
- Pro 的自动并发上限可达到 30,000，Enterprise 为 100,000。
- 单区域 burst concurrency 初始限制为每 10 秒 1,000 次并发扩展，超限可能出现 503 FUNCTION_THROTTLED。
- Fluid Compute 可以让同一实例并发处理多个 I/O 型请求，减少实例和冷启动。
- Function 应尽量靠近数据源部署。

这说明 Vercel Pro 不是 Friemi 近期最可能先触顶的部分。更可能先出现的是数据库查询、连接等待、实时消息模型和费用放大。

但 Vercel 的高 Function 并发也可能反向压垮数据库：前端计算层能快速扩出更多实例，每个实例都向同一个 PostgreSQL 发请求。如果没有 pooler、背压和请求合并，Function 扩展越快，数据库越容易过载。

Supabase 的实际边界

Supabase 不是“Pro 只有一个固定小数据库”。官方提供从 Micro 到 16XL 的计算规格，连接数和 pooler clients 随规格上升，也支持只读副本。

Supabase Realtime 当前项目配额：

计划	并发连接	消息/秒	Channel joins/秒
Free	200	100	100
Pro	500	500	500
Pro (关闭 spend cap)	10,000	2,500	2,500
Team	10,000	2,500	2,500
Enterprise	10,000+	2,500+	2,500+

这些是单项目配额，不等于平台技术极限。Supabase 公布的 Realtime benchmark 在不同集群配置下测试过 32,000 至 250,000 concurrent users，但官方同时明确要求每个项目根据自己的 payload、RLS、fan-out 和数据库配置自行压测。

对 Friemi 最重要的限制是：

- Postgres Changes 在高订阅人数下会为每个订阅者做权限检查，官方明确说明扩展性不如 Broadcast。
- Broadcast 更适合作为聊天、通知和游戏事件的实时传输层。
- Realtime 只负责传输，消息持久化和幂等仍需由 PostgreSQL 业务事务保证。
- 500 个连接是普通 Pro 的明确门槛；如果目标 CCU 接近 400，就应提前测试关闭 spend cap、Team/Enterprise 或替代实时服务。

官方资料：

- [Supabase Realtime limits](#)
- [Supabase Realtime benchmarks](#)
- [Supabase Realtime architecture](#)
- [Supabase Broadcast](#)
- [Supabase Realtime database changes](#)
- [Supabase Read Replicas](#)

17. 大型社交产品真正解决的是什么

大型产品没有一种“选对数据库后永远不用改”的架构。它们共同解决的是：

1. 不让每个客户端通过高频轮询制造全量读取。
2. 将持久化、实时分发、缓存、搜索、媒体和分析拆成不同负载。
3. 让热点按用户、频道、内容或时间范围分区。
4. 用事件和 read model 服务高频读取。
5. 在客户端维护局部状态，只传输变化。
6. 持续压测、观测尾延迟并按证据演进。

Slack：写入与实时分发分离

Slack 公开的实时消息架构包括：

- 客户端保持 WebSocket 连接接收实时事件。
- Webapp API 负责发送命令和持久化消息。
- Gateway Server 维护连接和订阅。
- Channel Server 按 channel ID 一致性哈希分配并负责 fan-out。
- Presence Server 使用内存状态，并且只推送屏幕中可见用户的 Presence。
- Gateway 部署在多个地理区域，使连接靠近用户。

映射到 Friemi：

- 发送消息继续走可靠 HTTP/Server Action 并写 PostgreSQL。
- 接收消息、角标、Presence 和狼人杀 revision 走一个实时连接。

- 不再让每位用户每 6 至 15 秒重新查询整个页面。
- Presence 只订阅当前页面真正展示的人，而不是全局扫描。

来源：[Slack Real-time Messaging](#)、[Slack Shared Channels](#)

Discord：Read State 是独立热路径

Discord 公开说明其 Read State 是“每用户、每频道”一条状态，包含需要原子更新和归零的计数；它使用内存缓存，并将持久化写入延迟/批量执行，而不是每次展示角标都扫描所有历史消息。

Discord 的消息存储还按 channel 和时间 bucket 分区，以限制单分区大小。它也遇到过热点分区和尾延迟，并从 MongoDB 演进到 Cassandra，之后又迁移到 ScyllaDB。

映射到 Friemi：

- ConversationPreference 和 ActivityRoomReadState 应发展为真正的 read model，直接保存 unread count、last sequence 和 mention count。
- 消息 ID/sequence 应支持 cursor 增量读取。
- 热门大群不能和低活跃会话使用完全相同的扫描方式。
- “永不迁移”不是现实目标；目标应是迁移时不重写产品逻辑。

来源：[Discord Read States](#)、[How Discord Stores Trillions of Messages](#)

Meta：缓存和分片在数据库之前

Meta 的 TAO 将缓存服务放在 MySQL 之前，并按 social graph shard 分布数据。大部分读取先在本地/区域缓存完成，同时专门处理突然爆红的热点对象。

映射到 Friemi：

- 首页活动、公开活动、分类和公告等公共数据应该优先命中缓存。
- 收藏、关注和参加状态作为小型用户 overlay，而不是让公共卡片查询整体动态化。
- 只有出现已证实热点时再引入 Redis 或分片，当前阶段不需要复制 TAO。

来源：[TAO: The power of the graph](#)、[Scaling memcache at Facebook](#)

抖音/ByteDance：交互反馈进入独立实时数据链路

ByteDance 的公开 Monolith 论文重点是将实时用户反馈送入在线训练，而不是让推荐请求直接执行所有分析工作。公开的 Flink 系统资料也说明实时流处理是独立基础设施。

映射到 Friemi：

- 点击、浏览、消息延迟和推荐行为应作为事件进入异步分析链路。
- 用户页面请求不应同步等待推荐训练、统计聚合或大量 Analytics INSERT。
- Friemi 当前规模不需要 Flink/Kafka 集群，但需要先把 analytics sink 与核心事务解耦。

来源：[Monolith paper](#)、[StreamShield paper](#)

当前不应该照搬的部分

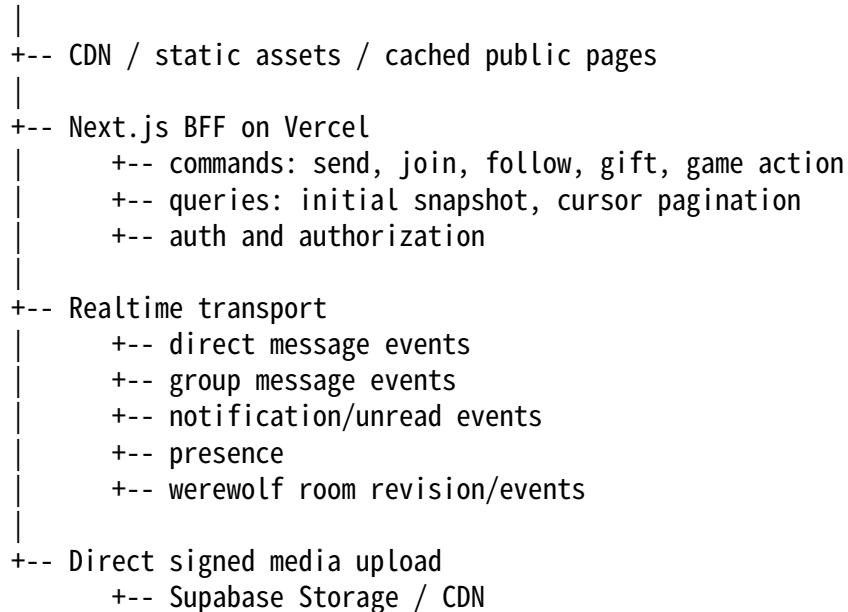
- 不立即拆几十个微服务。
- 不立即引入 Kubernetes。
- 不因为 Discord 使用 ScyllaDB 就放弃 PostgreSQL。
- 不为了未来推荐系统提前部署 Kafka/Flink。
- 不在缺少 100 CCU 基线时设计 100 万 CCU 架构。

大型系统的技术栈是多年真实瓶颈演进的结果。Friemi 现在应该复制其边界原则和压测方法，而不是复制最终组件数量。

18. Friemi 推荐的长期架构

18.1 总体分层

Web / Android WebView / iOS



PostgreSQL: source of truth + transactions + read models

Queue/Worker: push, thumbnails, analytics, cleanup, retries

Observability: traces, metrics, logs outside the business hot path

18.2 数据库是事实源，Realtime 不是事实源

消息、送礼、报名和狼人杀动作必须先完成可靠业务事务：

1. 校验用户与权限。
2. 使用 idempotency key 防止重复提交。
3. 在事务中写入业务记录和单调 sequence/revision。
4. 同事务写入 outbox event，或使用可证明可靠的数据库 Broadcast trigger。
5. 提交后向在线客户端发布事件。
6. 客户端断线重连后按 sequence/cursor 补齐遗漏。

Realtime 事件可以丢失或重复，业务记录不能丢失或重复。客户端必须按 event ID/revision 幂等应用。

18.3 只为基础设施热点建立边界

不需要给每个 Prisma model 创建 repository。只为未来可能替换的四个热点建立清晰适配边界：

- RealtimePublisher: Supabase Broadcast、Vercel WebSocket 或独立 Gateway。
- PresenceStore: PostgreSQL、Redis TTL 或 Realtime Presence。
- AnalyticsSink: PostgreSQL sampling、外部 analytics 或 queue。
- MediaStorage: Supabase Storage、S3 compatible storage 或其他对象存储。

业务层只依赖“发布事件、读取 Presence、记录 Analytics、保存媒体”这些能力，不在页面组件里散落供应商 API。

Friemi 当前身份认证使用 Clerk，而不是 Supabase Auth。接入私有 Realtime channel 前必须完成一个独立授权 POC，验证短期 token、channel topic 权限、过期刷新、撤权和服务端 RLS，且不能把 Supabase service role key 下发给客户端。

18.4 图片必须绕开 Function 数据中转

当前上传路径是：

浏览器 -> Vercel Function 读取完整 FormData/Buffer -> Supabase Storage

多人同时上传时，这会同时消耗 Vercel 请求体传输、Function 内存、执行时长和 Supabase 上传带宽。建议改为：

浏览器 -> Friemi 获取短期 signed upload token

浏览器 -> Supabase Storage 直接上传

Friemi -> 校验完成状态并写入业务记录

超过 6 MB 或弱网场景使用 TUS resumable upload。Supabase 官方支持 signed upload token、进度、断点续传和 direct storage hostname。

来源：[Supabase Resumable Uploads](#)、[Supabase Storage CDN](#)

19. 平台演进路线

以下分级使用峰值同时在线用户（CCU），不是注册用户数。具体升级点必须由压测和成本数据确认。

阶段	参考 CCU	推荐形态	重点
A	0-100	Vercel + Supabase，优化后的单体	消除轮询、增量消息、索引、缓存、监控
B	100-500	Vercel Pro + Supabase Pro/Micro 或 Small + Realtime	真实压测、预算告警、直接上传、异步任务
C	500-5,000	Pro no-spend-cap/Team 或替代实时服务；数据库按指标升至 Small/Medium	read model、queue、Presence TTL、热点缓存
D	5,000-10,000	提前联系平台支持；可能增加 read replica、独立 worker/realtime service	多区域、故障演练、成本/用户、容量预留
E	10,000+	Enterprise/custom 或专用实时 Gateway；数据库读写分离	多区域容灾、分片评估、专职 SRE/数据平台

这张表不是容量承诺。例如 500 人同时刷高 fan-out 大群，比 5,000 人静默浏览缓存页面更重。

什么时候升级 Supabase Compute

满足以下条件之一并持续出现，再升级：

- 查询和索引已经优化，DB CPU 仍持续超过安全水位。
- 内存不足、cache hit 下降或发生 swap/高 I/O。
- pool wait/P2024 在合理连接配置下仍出现。
- 写入或 vacuum/maintenance 与在线请求互相影响。
- 发布活动前的压测无法满足 P95/P99。

不要因为单个慢页面直接升级数据库；先确认它不是 20 条串行 SQL 或无界查询。

什么时候增加 Read Replica

- 读流量长期占 80% 以上并压制主库写入。
- analytics/report/export 可以接受 replication lag。
- 已经使用至少 Small compute，并且 query/index/cache 均已优化。
- 需要为其他地区提供低延迟只读内容。

聊天刚发送的消息、余额、报名状态和游戏状态通常要求 read-your-write，不应盲目读 replica。

什么时候从 Vercel 拆出服务

不是“用户一多就全部迁走”，而是按负载拆：

- 长连接需要更稳定的 durable state 或超过当前 WebSocket/Realtime 配额。
- 后台任务需要持续运行、消费队列或长时间处理。
- Function 成本持续高于容器服务，且流量已稳定可预测。
- 需要自定义网络、固定 egress、专用 CPU 或精细 autoscaling。

此时仍可保留 Next.js 页面和 API BFF 在 Vercel，只把 realtime gateway、worker 或媒体处理迁到长期运行的服务。

20. 避免未来被平台锁死

完全避免迁移不现实，但可以避免“迁移等于重写”。

数据库

- 保持 PostgreSQL 和 Prisma migration 为事实源。
- 核心表优先使用标准 PostgreSQL 类型和约束。
- Supabase-specific trigger、RLS、Realtime publication 集中在独立 migration 和 adapter。
- 定期执行 pg_dump 并在空数据库做 restore drill。
- 不只验证备份生成，还要验证可以恢复和切换连接串。

Supabase 支持标准数据库 dump，也开源并支持 self-host；这不代表自建成本低，但意味着退出路径存在。

Next.js/Vercel

- 不在业务代码中假设只有 Vercel 才能运行。
- 环境配置集中管理，不在页面内读取供应商特有变量。
- 对 background jobs、cache 和 realtime 使用 adapter。
- 保留 Docker/self-host smoke test。
- 多实例自建时需要共享 cache 和 tag invalidation，不能简单启动多个 next start 就认为完成迁移。

Next.js 官方说明其最低运行要求只是 Node.js server，也提供多实例 cache coordination 指南。

来源：[Next.js Self-Hosting](#)、[Deploying Next.js to platforms](#)、[Supabase Self-Hosting](#)

Storage

- 数据库只保存 object key、bucket 和必要 metadata，不把完整供应商 URL 当不可变 ID。
- 展示 URL 由 storage adapter 生成。
- 文件路径使用不可变版本，不覆盖旧 URL。
- 定期导出对象清单，验证跨 S3-compatible provider 的复制流程。

认证

Friemi 当前广泛依赖 Clerk。认证也是容量、成本和可迁移性的一部分：

- 业务表使用自己的 UserProfile.id，不要让 Clerk user ID 成为所有业务外键。
- Clerk 只负责身份认证，授权规则继续由 Friemi 数据库控制。

- token verification 和用户快照读取应减少重复数据库访问。
- 在平台容量验收中单独记录 Clerk latency、rate limit 和费用。

21. 当前与目标请求模型对比

优化前空闲登录用户

优化前每名可见页面用户的固定 HTTP 请求率约为：

2 个未读接口 / 15 秒 + 1 个 Presence / 60 秒
= 0.15 RPS / 在线用户

CCU	仅后台固定 HTTP RPS	每分钟请求
100	15	900
500	75	4,500
1,000	150	9,000
10,000	1,500	90,000

这还没有包含页面访问、消息、图片和狼人杀。

两轮低风险优化后

不计 focus、页面切换和业务事件触发的即时校准，每名持续可见用户的固定请求率约为：

1 个聚合未读接口 / 45 秒 + 1 个 Presence / 90 秒
= 0.033 RPS / 在线用户

CCU	仅后台固定 HTTP RPS	每分钟请求
100	3.33	200
500	16.67	1,000
1,000	33.33	2,000
10,000	333.33	20,000

与优化前模型相比，角标和 Presence 合计固定 HTTP 请求约减少 78%。这是静态模型，必须用预览环境同时时间窗口指标验证。

目标事件模型

目标状态下：

- 每位在线用户保持 1 条 Realtime 连接。
- 没有新事件时不产生数据库轮询。
- Presence 使用连接状态/TTL，不每分钟更新主表。
- HTTP 主要用于页面初始快照、用户命令和断线后的 cursor 补齐。
- 单个业务事件的成本主要与真实接收人数相关，而不是“所有客户端固定轮询”。

这会把容量问题从不可控的固定请求税，转变为可以按真实业务事件计费 and 扩展的模型。

22. 充值前必须完成的研究与验证

- ☒ 核对 Vercel Function 自动扩展、Fluid Compute 和 Pro concurrency 官方边界。
- ☒ 核对 Supabase compute、连接数、Realtime 和 read replica 官方边界。

- ☒ 分析 Slack、Discord、Meta 和 ByteDance 的公开架构原则。
- ☒ 检查 Friemi 当前聊天、未读、Presence、狼人杀和上传链路。
- ☒ 明确 Vercel/Supabase 的退出和部分拆分路径。
- ☐ 在 Vercel Dashboard 导出当前 Hobby 实际 Function duration、invocation、cold start 和 error。
- ☐ 在 Supabase Dashboard 导出当前 Nano CPU、memory、I/O、connections、cache hit 和 slow queries。
- ☐ 确认当前两个平台的实际 plan、region、spend cap 和 Fluid Compute 设置。
- ☐ 完成 Clerk 与 Supabase private Realtime channel 的授权 POC。
- ☐ 完成当前免费配置 20 / 50 / 100 CCU 基线。
- ☐ 完成 P0 请求治理后，在同一免费配置复测。
- ☐ 建立短期 Pro 测试环境，在相同脚本下再次复测。
- ☐ 比较三组结果：原始 Free、优化后 Free、优化后 Pro。
- ☐ 计算每 1,000 DAU、每 100 CCU 和每 100 万消息的月成本。
- ☐ 根据指标决定 Vercel Pro、Supabase Pro/compute 和 Realtime 方案。

只有这样才能回答“充值能提升多少”，而不是把代码优化和套餐提升混在一起。

23. 当前推荐决策

现在不迁离 Vercel/Supabase

理由：

- Vercel Pro 的 Function 并发能力明显高于 Friemi 当前已验证规模。
- Supabase 可以纵向扩容、增加 Realtime 配额和 read replica。
- PostgreSQL、Prisma、Next.js 和 S3-compatible Storage 都有迁移路径。
- 当前最明确的问题来自应用请求模型，换平台会把相同问题带到新环境。
- 自建基础设施会立即引入高可用、备份、升级、安全、监控和 24/7 运维责任。

现在也不直接依赖“升级套餐解决”

先完成：

1. 未读请求合并和降频。
2. 私聊/群聊增量更新，取消周期路由刷新。
3. 狼人杀 revision/事件同步。
4. Presence TTL。
5. 图片 signed direct upload。
6. Analytics 与业务热路径解耦。
7. 100 CCU 可重复压测。

然后再开 Pro 做同脚本对照。若优化后 Free 已满足延迟但缺少生产可靠性、备份或商业使用资格，升级是运营与可靠性决策；若优化后仍由 CPU/内存/连接限制导致 P95/P99 不达标，升级才是明确的性能决策。